C# 멀티스레드 채팅 프로그램 – 프로젝트 구조 및 요구사항 분석서

Oct 27, 2026 · @AI2_반재영

구분	내용
팀명	다같이채팅
팀장	박은정
팀원	반재영, 장선화

1. 프로젝트 개요

C#/.NET 기반의 TCP 소켓 채팅 프로그램으로, 서버는 클라이언트 1명당 1개의 수신 스레드를 두어 다수 사용자의 동시 채팅을 처리한다. 확장 범위로 채팅방(룸), 회원가입/로그인(DB), 파일 전송까지 포함한다.

컨셉: 사내 메신저. 서버 PC 1대를 사내망에 두고, 직원은 서버의 내부 IP(예: 192.168.0.15:9000) 하나로 접속한다. 클라이언트끼리는 직접 연결하지 않으므로 서로의 IP는 필요 없다. 가입은 사내 이메일과 사원번호로 하고, 로그인은 사원번호로 한다.

목적

- 멀티스레드 환경에서의 소켓 통신, 동기화(lock), 스레드 안전 컬렉션 사용법을 실습한다.
- 서버–클라이언트 구조와 자체 통신 프로토콜을 설계·구현한다.

기술 스택

구분	기술
언어/런타임	C# 14, .NET 10 (LTS), Visual Studio 2026
서버	콘솔 앱 (또는 WinForms 관리 화면), `TcpListener`, `Thread` / `Task`
클라이언트	WinForms, `TcpClient`, `NetworkStream`
동시성	`lock`, `ConcurrentDictionary`, `BlockingCollection`, `CancellationToken`
직렬화	`System.Text.Json` (길이 헤더 + JSON 본문)

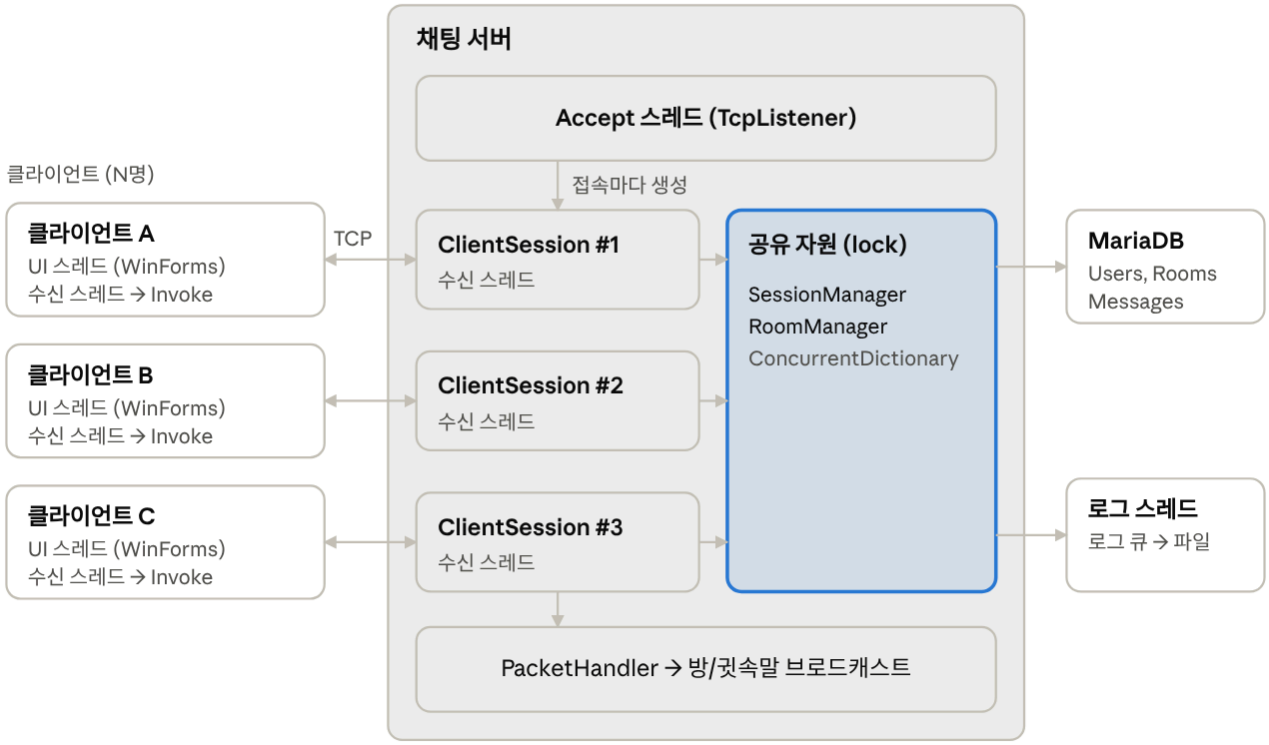
구분	기술
DB	MariaDB 11.x LTS (InnoDB, utf8mb4) + `MySqlConnector` (NuGet)
보안	비밀번호 해시 (`Rfc2898DeriveBytes` / PBKDF2 + Salt)
협업/형상관리	Git, GitHub

범위 밖: 음성·영상 통화, 메시지 종단간 암호화, 모바일 클라이언트.

2. 시스템 아키텍처

서버는 Accept 스레드 1개, 클라이언트별 수신 스레드 N개, 로그 스레드 1개로 동작한다. 여러 스레드가 동시에 접근하는 세션·방 목록이 동기화 대상이다.

> **클라이언트 1명당 수신 스레드 1개, 공유 자원은 lock으로 보호**



서버 스레드 모델 · Accept 1 + 세션별 수신 스레드 N + 로그 1

수신 스레드는 패킷을 읽어 PacketHandler에 넘기고, 브로드캐스트는 공유 목록을 조회해 대상 세션에 전송한다. 클라이언트는 UI 스레드와 수신 스레드를 분리해 화면이 멈추지 않게 한다.

3. 솔루션 폴더 구조

하나의 솔루션에 Server, Client, Common(공유 라이브러리), Tests 4개 프로젝트를 두고, 패킷 정의는 Common에만 둔다.

```
ChatApp.slnx
|
├─ ChatApp.Common/                # 서버·클라이언트 공유 (Class Library)
|   ├─ Protocol/
|   |   ├─ PacketType.cs           # enum: LoginReq, ChatMsg, Whisper, Admin*
...
|   |   ├─ Packet.cs               # 공통 패킷 모델 (Type, Sender, Body ...)
|   |   └─ PacketSerializer.cs     # 길이 헤더 + JSON 직렬화/역직렬화
|   ├─ Models/
|   |   ├─ UserInfo.cs             # 사번, 이름, 부서, 직급, 역할, 상태
|   |   └─ RoomInfo.cs
|   └─ Utils/
|       └─ NetworkHelper.cs        # ReadPacket, WritePacket (Stream 기준 →
SslStream에도 사용)
|
├─ ChatApp.Server/                # 서버 (Console)
|   ├─ Program.cs
|   ├─ appsettings.json            # DB·LDAP 연결 정보, 인증서 경로 (Git 제외)
|   ├─ Core/
|   |   ├─ ChatServer.cs           # TcpListener, Accept 루프 (Accept 스레드)
|   |   ├─ ClientSession.cs        # 클라이언트 1명 = 수신 스레드 1개
|   |   └─ SessionManager.cs       # ConcurrentDictionary<사번, Session>
|   ├─ Security/
|   |   ├─ TlsHelper.cs            # 인증서 로드, SslStream 인증
(AuthenticateAsServer)
|   |   └─ PasswordHasher.cs       # PBKDF2 + Salt
|   ├─ Services/
|   |   ├─ PacketHandler.cs        # PacketType별 분기 처리
|   |   ├─ AuthService.cs          # 회원가입/로그인, 시도 제한 (LOCAL)
|   |   ├─ LdapAuthService.cs      # 사내 계정 연동 로그인 (LDAP)
|   |   ├─ AdminService.cs         # 승인·정지·퇴사·공지·강퇴 + 권한 검사
|   |   ├─ RoomManager.cs          # 채팅방 생성/입장/퇴장
|   |   ├─ FileTransferService.cs  # 파일 중계
|   |   └─ RetentionWorker.cs      # 보관 기간 지난 기록 정리 (백그라운드 스레드)
|   ├─ Data/
|   |   ├─ Db.cs                   # MySqlConnection 생성 (요청마다 새 연결)
|   |   ├─ UserRepository.cs
|   |   ├─ MessageRepository.cs
|   |   └─ AuditRepository.cs      # 감사 로그
|   └─ Logging/
|       └─ ServerLogger.cs         # 로그 큐 + 전용 로그 스레드
```

```
|
├─ ChatApp.Client/                # WinForms 클라이언트
|   ├─ Program.cs
|   ├─ Network/
|   |   ├─ ChatClient.cs           # TcpClient + SslStream, 수신 스레드, 송신 메
서드
|   |   └─ ClientPacketHandler.cs  # 수신 패킷 → 이벤트 발생
|   └─ Forms/
|       ├─ LoginForm.cs            # 로그인 / 회원가입
|       ├─ LobbyForm.cs            # 채팅방 목록, 부서별 접속자 목록
|       ├─ ChatRoomForm.cs         # 채팅창, 귓속말, 파일 전송
|       └─ AdminForm.cs            # 관리자 전용: 계정 관리, 기록·감사 로그 조회
|
├─ ChatApp.ConsoleClient/         # 1~2단계 검증용 콘솔 클라이언트
|
└─ ChatApp.Tests/                 # xUnit 단위/부하 테스트
    ├─ PacketSerializerTests.cs
    └─ MultiClientLoadTests.cs
```

클라이언트의 수신 스레드는 UI를 직접 건드리지 않고, 이벤트 → `Control.Invoke` (또는 `BeginInvoke`)로 UI 스레드에 전달한다.

4. 기능 요구사항

총 22개 기능 중 상(필수) 13개를 1차 구현 목표로 한다. 우선순위: 상 = 필수, 중 = 권장, 하 = 선택.

ID	구분	기능명	상세 내용	우선순위
SV-01	서버	서버 시작/종료	지정 IP·포트에서 Listen 시작, 종료 시 전체 세션을 안전하게 닫는다	상
SV-02	서버	다중 접속 처리	Accept 스레드가 접속을 받고, 클라이언트마다 수신 스레드를 생성한다	상
SV-03	서버	세션 관리	접속자 목록을 스레드 안전 컬렉션으로 관리하고 비정상 종료를 감지해 제거한다	상
SV-04	서버	서버 로그	접속/해제/오류/메시지를 시간과 함께 화면과 파일에 기록한다	중
SV-05	서버	관리자 기능	공지 방송, 특정 사용자 강제 퇴장	하

ID	구분	기능명	상세 내용	우선순위
AU-01	회원	회원가입	사원번호·사내 이메일·이름·부서·직급·비밀번호로 가입한다. 사내 도메인(예: @회사.kr) 이메일만 허용하고, 사원번호·이메일 중복을 확인하며, 비밀번호는 해시해 저장한다	상
AU-02	회원	로그인	사원번호와 비밀번호로 로그인한다. 성공 시 세션을 등록하고, 이미 접속 중인 사원번호는 거부한다. 채팅에는 “이름 직급”(예: 홍길동 대리)으로 표시한다	상
AU-03	회원	로그아웃	로그아웃 시 세션·채팅방에서 제거하고 퇴장을 알린다	상
RM-01	채팅방	방 목록 조회	로비에서 방 이름·인원을 조회하고 변경 시 자동 갱신한다	상
RM-02	채팅방	방 생성	방 이름, 최대 인원, (선택) 비밀번호를 입력해 생성한다	상
RM-03	채팅방	방 입장/퇴장	입·퇴장 시 방 인원에게 알림, 마지막 인원 퇴장 시 방을 삭제한다	상
RM-04	채팅방	방장 권한	방장 위임, 강퇴	하
CH-01	채팅	방 채팅	같은 방 인원 전체에게 메시지를 브로드캐스트한다	상
CH-02	채팅	귓속말	지정한 사용자에게만 전송(`/w 아이디 내용` 또는 목록 더블클릭)	상
CH-03	채팅	접속자 목록	접속자 목록을 부서별로 묶어 이름·직급과 함께 실시간 표시한다	상
CH-04	채팅	입·퇴장 시스템 메시지	“OO님이 입장하셨습니다” 등을 다른 색으로 표시한다	중
CH-05	채팅	대화 기록 저장/조회	메시지를 DB에 저장하고 입장 시 최근 N개를 불러온다	중
FT-01	파일	파일 전송	파일을 청크 단위로 나눠 서버 중계로 전송, 수신자는 수락/거절 가능	중
FT-02	파일	전송 진행률	별도 스레드에서 전송하며 ProgressBar로 진행률을 표시한다	중
FT-03	파일	크기 제한	최대 크기(예: 50MB)와 허용 확장자를 제한한다	하

ID	구분	기능명	상세 내용	우선순위
CL-01	클라이언트	연결 상태 표시	연결 끊김 감지 및 재접속 안내	상
CL-02	클라이언트	입력 편의	Enter 전송, 빈 메시지 차단, 자동 스크롤	중

4-1. 확장 기능 요구사항 (보안·관리자·사내 계정)

기본 기능 완료 후 추가하는 9개 기능이다. AD-04는 기존 SV-05(공지·강제 퇴장)를 관리자 기능으로 옮긴 것이다.

ID	구분	기능명	상세 내용	우선순위
SC-01	보안	TLS 통신 암호화	`NetworkStream`을 `SslStream`으로 감싸 모든 패킷을 암호화하고, 클라이언트는 서버 인증서를 검증한다	중
SC-02	보안	로그인 시도 제한	비밀번호 5회 연속 실패 시 해당 계정을 10분간 잠근다	중
AD-01	관리자	관리자 권한	계정에 역할(USER/ADMIN)을 두고, 관리자 화면과 기능은 ADMIN만 사용한다	중
AD-02	관리자	가입 승인	신규 가입자는 승인 대기(PENDING) 상태이며, 관리자 승인 후 로그인할 수 있다	중
AD-03	관리자	계정 정지·퇴사 처리	정지(SUSPENDED)·퇴사(RESIGNED) 처리 시 접속 중이면 즉시 연결을 끊고 재로그인을 막는다	중
AD-04	관리자	공지·강제 퇴장	전체 공지를 보내고 특정 사용자를 강제 퇴장시킨다	중
AD-05	관리자	대화 기록 조회·보관 기간	기간·사용자별로 대화 기록을 검색하고, 보관 기간(예: 180일)이 지난 기록은 정리 스레드가 하루 1회 삭제한다	하
AD-06	관리자	감사 로그	로그인 성공·실패와 관리자 조치(승인·정지·강퇴 등)를 시간·IP와 함께 기록한다	하

ID	구분	기능명	상세 내용	우선순위
AU-04	회원	사내 계정(LDAP/AD) 연동	사원번호·비밀번호를 사내 디렉터리 서버에 LDAP으로 검증하고, 첫 로그인 시 이름·부서·직급을 자동 등록한다. 설정으로 자체 계정(LOCAL)과 선택한다	하

사내 계정 연동은 집에 사내 AD 서버가 없으므로, Docker로 OpenLDAP 테스트 서버를 띄워 구현·시연한다. C#에서는 `System.DirectoryServices.Protocols`를 쓴다.

5. 비기능 요구사항

핵심은 동시성(공유 자원 보호)과 안정성(한 클라이언트 오류가 서버 전체를 멈추지 않음)이다. 수치는 실습 환경 기준 목표값이다.

ID	항목	요구사항
NF-01	동시 접속	동시 접속 100명에서 정상 동작
NF-02	응답 속도	로컬 네트워크에서 메시지 전달 100ms 이내
NF-03	스레드 안전	세션·방 목록은 `ConcurrentDictionary` 또는 `lock`으로 보호, 브로드캐스트는 목록 스냅샷을 순회
NF-04	송신 동기화	한 소켓에 여러 스레드가 동시에 쓰지 않도록 세션별 송신 lock 또는 송신 큐 사용
NF-05	장애 격리	클라이언트 스레드 예외는 try-catch로 해당 세션만 종료
NF-06	자원 해제	연결 종료 시 Socket/Stream Dispose, 스레드는 `CancellationToken`으로 종료
NF-07	패킷 무결성	TCP 분할/병합 문제를 길이 헤더(4바이트)로 해결
NF-08	하트비트	30초마다 PING/PONG, 90초 무응답 시 연결 해제
NF-09	보안	비밀번호 평문 저장 금지(PBKDF2 + Salt), SQL은 파라미터 바인딩
NF-10	UI 응답성	네트워크·파일 I/O는 UI 스레드에서 하지 않음, UI 갱신은 `Invoke`
NF-11	유지보수	패킷 정의는 Common 프로젝트에만 두고, 핸들러는 PacketType별로 분리

확장 기능에 따른 보안 요구사항:

ID	항목	요구사항
NF-12	통신 암호화	TLS 1.2 이상. 개발·시연에서는 자체 서명 인증서를 쓰고, 클라이언트는 인증서 지문(thumbprint)으로 서버를 확인
NF-13	권한 검사	관리자 패킷은 서버가 매번 DB의 role을 확인. 클라이언트에서 버튼을 숨기는 것만으로 막지 않음
NF-14	계정 정보 보호	LDAP 계정의 비밀번호는 저장하지 않고 검증에만 사용, LDAP 통신은 LDAPS(636 포트)로 암호화

6. 통신 프로토콜

모든 패킷은 `[본문 길이 4바이트(int32)] + [UTF-8 JSON 본문]` 형식이며, 수신 측은 길이만큼 정확히 읽은 뒤 역직렬화한다.

```
{
  "type": "CHAT_MSG",
  "sender": "user01",
  "target": "room-3",
  "body": "안녕하세요",
  "timestamp": "2026-10-27T14:30:00"
}
```

PacketType	방향	용도
REGISTER_REQ / REGISTER_RES	C→S / S→C	회원가입 요청 / 결과
LOGIN_REQ / LOGIN_RES	C→S / S→C	로그인 요청 / 결과
LOGOUT	C→S	로그아웃
ROOM_LIST_REQ / ROOM_LIST	C→S / S→C	방 목록 요청 / 전달
ROOM_CREATE / ROOM_JOIN / ROOM_LEAVE	C→S	방 생성 / 입장 / 퇴장
ROOM_RES	S→C	방 요청 결과(성공/실패 사유)
USER_LIST	S→C	접속자 목록 갱신
CHAT_MSG	C↔S	방 채팅 메시지

PacketType	방향	용도
WHISPER	C↔S	귓속말
SYSTEM_MSG	S→C	입·퇴장, 공지 등 시스템 알림
FILE_OFFER / FILE_ACCEPT / FILE_REJECT	C↔S	파일 전송 제안 / 수락 / 거절
FILE_CHUNK / FILE_END	C↔S	파일 데이터 청크(Base64 또는 바이너리) / 전송 완료
PING / PONG	C↔S	연결 유지 확인
ERROR	S→C	오류 코드와 메시지

파일 청크는 64KB 단위를 권장하며, 대용량 전송이 채팅을 막지 않도록 송신 큐에서 채팅 패킷과 번갈아 보낸다.

확장 기능용 패킷 타입:

PacketType	방향	용도
ADMIN_USER_LIST_REQ / ADMIN_USER_LIST	C→S / S→C	전체 계정 목록(상태 포함)
ADMIN_APPROVE / ADMIN_SUSPEND / ADMIN_RESIGN	C→S	가입 승인 / 정지 / 퇴사 처리
ADMIN_NOTICE / ADMIN_KICK	C→S	공지 / 강제 퇴장
ADMIN_LOG_QUERY / ADMIN_LOG_RESULT	C→S / S→C	대화 기록·감사 로그 조회
FORCE_LOGOUT	S→C	정지·퇴사·강제 퇴장 사유를 알리고 연결 종료

TLS를 적용해도 패킷 형식은 그대로다. `NetworkStream` 대신 이를 감싼 `SslStream`에 같은 방식으로 읽고 쓴다.

7. DB 설계

DB는 MariaDB를 쓰고, 테이블은 users, rooms, messages, file_logs 4개다. 1:N 구조에서 여러 세션 스레드가 동시에 읽고 쓰므로, 쓰기가 한 번에 하나로 제한되는 SQLite보다 행 단위 잠금(InnoDB)을 지원하는 MariaDB가 적합하다. 채팅방의 현재 인원은 서버 메모리에서 관리하고, DB에는 기록만 남긴다.

```
CREATE DATABASE chatapp
  DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  -- 한글·이모지 저장
USE chatapp;

-- 서버 프로그램 전용 계정 (root 사용 금지)
CREATE USER 'chatapp'@'localhost' IDENTIFIED BY '비밀번호';
GRANT SELECT, INSERT, UPDATE, DELETE ON chatapp.* TO 'chatapp'@'localhost';

CREATE TABLE users (
  employee_no         VARCHAR(20)  PRIMARY KEY,           -- 사원번호 = 로그인
아이디
  email               VARCHAR(100) NOT NULL UNIQUE,       -- 사내 도메인만 허용
  name                VARCHAR(30)  NOT NULL,
  department          VARCHAR(30)  NOT NULL,
  position            VARCHAR(20)  NOT NULL,              -- 직급
  role                ENUM('USER','ADMIN') NOT NULL DEFAULT 'USER',
  status              ENUM('PENDING','ACTIVE','SUSPENDED','RESIGNED') NOT
NULL DEFAULT 'PENDING',
  auth_source         ENUM('LOCAL','LDAP') NOT NULL DEFAULT 'LOCAL',
  password_hash       VARCHAR(100) NULL,                  -- LDAP 계정은 NULL
  salt                VARCHAR(50)  NULL,
  failed_login_count  TINYINT UNSIGNED NOT NULL DEFAULT 0,
  locked_until        DATETIME     NULL,                  -- 5회 실패 시 잠금 해
제 시각
  created_at          DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  last_login_at       DATETIME     NULL
) ENGINE=InnoDB;

CREATE TABLE rooms (
  room_id     INT AUTO_INCREMENT PRIMARY KEY,
  room_name   VARCHAR(50)      NOT NULL,
  owner_no    VARCHAR(20)      NULL,
  max_users   TINYINT UNSIGNED NOT NULL DEFAULT 10,
  created_at  DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (owner_no) REFERENCES users(employee_no) ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE messages (
  message_id   BIGINT AUTO_INCREMENT PRIMARY KEY,
  room_id      INT          NULL,
  sender_no    VARCHAR(20)  NOT NULL,
  receiver_no  VARCHAR(20)  NULL,                         -- 귓속말일 때만 값
  content      VARCHAR(500) NOT NULL,
  msg_type     ENUM('CHAT','WHISPER','SYSTEM') NOT NULL,
  sent_at      DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  INDEX idx_room_time (room_id, sent_at),                 -- 입장 시 최근 N개 조
```

```
회용
  INDEX idx_sent_at (sent_at),                              -- 보관 기간 정리·기간
검색용
  FOREIGN KEY (room_id)   REFERENCES rooms(room_id) ON DELETE SET NULL,
  FOREIGN KEY (sender_no) REFERENCES users(employee_no)
) ENGINE=InnoDB;

CREATE TABLE file_logs (
  file_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
  sender_no    VARCHAR(20)  NOT NULL,
  receiver_no  VARCHAR(20)  NOT NULL,
  file_name    VARCHAR(255) NOT NULL,
  file_size    BIGINT       NOT NULL,
  status       ENUM('SENT','REJECTED','FAILED') NOT NULL,
  sent_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (sender_no) REFERENCES users(employee_no)
) ENGINE=InnoDB;

CREATE TABLE audit_logs (
  log_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
  actor_no    VARCHAR(20)  NULL,                            -- 행위자 (로그인 실패 시
입력한 사번)
  action      VARCHAR(30)  NOT NULL,                        -- LOGIN_OK,
LOGIN_FAIL, APPROVE, SUSPEND, RESIGN, KICK, NOTICE
  target_no   VARCHAR(20)  NULL,                            -- 조치 대상
  detail      VARCHAR(255) NULL,
  client_ip   VARCHAR(45)  NULL,                            -- IPv6 길이까지
  created_at  DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  INDEX idx_created (created_at)
) ENGINE=InnoDB;

-- 최초 관리자는 가입 후 SQL로 직접 지정
-- UPDATE users SET role = 'ADMIN', status = 'ACTIVE' WHERE employee_no =
'A0001';
```

멀티스레드 DB 접근 규칙

- `MySqlConnection`은 스레드 간에 공유하지 않는다. 요청마다 `using var conn = new MySqlConnection(connStr)` 로 열고 닫는다. 연결 풀링이 기본으로 켜져 있어 비용이 작다.
- 모든 SQL은 파라미터(`@userId`)로 값을 넘긴다(SQL Injection 방지, NF-09).
- 회원가입은 `user_id` PRIMARY KEY 제약으로 중복을 막는다. 두 스레드가 동시에 같은 아이디로 가입해도 한쪽은 중복 키 오류(1062)로 실패한다.
- 연결 문자열(비밀번호 포함)은 코드에 쓰지 않고 `appsettings.json` 또는 환경변수에 두며, 해당 파일은 `.gitignore`에 추가한다.

8. 개발 단계 및 테스트 계획

핵심 통신을 먼저 완성하고 기능을 쌓아 올리는 8단계로 진행한다. 주차는 예시이며 실제 기간에 맞게 조정한다.

1. **(1주차) 설계** — 요구사항 확정, 패킷 정의, 솔루션 생성, Git 저장소 준비
2. **(2주차) 핵심 통신** — Accept 스레드, 클라이언트별 수신 스레드, 길이 헤더 송수신, 전체 채팅(콘솔로 먼저 검증)
3. **(3주차) 클라이언트 GUI + 기본 기능** — WinForms 화면, 접속자 목록, 귓속말, 입·퇴장 알림
4. **(4주차) 회원/DB + 채팅방** — 사원번호 회원가입·로그인(MariaDB), 방 생성·입장·퇴장, 대화 기록 저장
5. **(5주차) 파일 전송 + 안정화** — 청크 전송, 진행률, 하트비트, 예외 처리
6. **(6주차) 보안·관리자** — TLS 암호화, 로그인 시도 제한, 관리자 권한·가입 승인·정지/퇴사 처리·공지·강퇴, 감사 로그
7. **(7주차) 사내 계정 연동 + 기록 관리** — Docker OpenLDAP 테스트 서버, LDAP 로그인, 대화 기록 조회·보관 기간 정리
8. **(8주차) 테스트·문서화** — 아래 테스트 수행, 버그 수정, 발표 자료

테스트 ID	종류	시나리오	기대 결과
T-01	단위	패킷 직렬화 후 역직렬화	원본과 동일
T-02	단위	2개 패킷을 붙여서 / 1개를 쪼개서 수신	패킷 경계를 정확히 복원
T-03	기능	회원가입 시 중복 아이디	실패 메시지 표시
T-04	기능	같은 아이디로 중복 로그인	두 번째 로그인 거부
T-05	기능	방 A의 채팅	방 B 사용자에게 보이지 않음
T-06	기능	귓속말 전송	대상자와 보낸 사람만 표시
T-07	예외	클라이언트 프로세스 강제 종료	서버 유지, 퇴장 알림, 목록에서 제거
T-08	예외	대용량 파일 전송 중 채팅	채팅 지연 없이 전송 완료
T-09	부하	더미 클라이언트 100개가 초당 1건씩 전송	메시지 유실·예외 없음
T-10	부하	접속/해제 반복 1,000회	스레드·메모리 누수 없음

확장 기능 테스트:

테스트 ID	종류	시나리오	기대 결과
T-11	보안	Wireshark로 채팅 패킷 캐처 (TLS 적용 전/후 비교)	적용 후에는 내용을 읽을 수 없음
T-12	보안	일반 사용자가 관리자 패킷을 직접 전송	서버가 거부하고 감사 로그에 기록
T-13	보안	비밀번호 5회 연속 오류	10분간 로그인 잠금
T-14	관리자	승인 대기 계정으로 로그인	“관리자 승인 대기” 안내 후 거부
T-15	관리자	접속 중인 사용자를 퇴사 처리	즉시 연결 종료, 재로그인 불가
T-16	연동	LDAP 계정 첫 로그인	users에 이름·부서·직급 자동 등록

T-11의 TLS 적용 전후 Wireshark 비교 화면은 발표 자료에 넣기 좋다.